

ROADRESQ

Production Operations / Runbook

Version 1.0 • Production Operations, Incident Response & Recovery

Audience: DevOps • Backend • Frontend • QA • Security • Support • Operations

Purpose: A practical operational guide for running RoadResQ safely in production

DETECT → TRIAGE → MITIGATE → VERIFY → RECOVER → LEARN

Document Control

Field	Value
Document	Production Operations / Runbook
Product	RoadResQ
Version	1.0
Status	Production operations baseline
Audience	Engineering, DevOps, QA, Security, Support and Operations
Related documents	Deployment & DevOps Plan • System Architecture • Security Threat Model • Testing & QA Plan • API Specification

1. Runbook Purpose

This runbook defines the standard operating procedures required to keep RoadResQ available, secure and operational in production. It covers routine operations, deployments, health verification, incidents, service degradation, database and queue operations, realtime tracking, payments, backups, security events and recovery.

This is an operational control document. It should be kept synchronized with the actual infrastructure, deployment tooling and application behavior.

2. Operating Principles

- Protect customer safety and booking integrity before optimizing performance.
- Do not make untracked production changes.
- Prefer reversible actions and documented procedures.
- Use the least privileged account necessary.
- Treat payment and financial records as authoritative business data; never fix them with destructive shortcuts.
- Preserve logs, timestamps, request IDs and audit evidence during incidents.
- Communicate clearly when an incident affects customers or providers.
- After major incidents, convert lessons into concrete engineering actions.

3. Production Components

Component	Operational responsibility
Web / Next.js	Customer, provider and admin web experience
FastAPI API	Core business APIs and authorization
Worker	Background jobs, notifications, retries and reconciliation
WebSocket / realtime	Booking and provider location/status events
PostgreSQL/PostGIS	Transactional and geospatial source of truth
Redis	Cache, rate limiting, queues, short-lived coordination
Object storage	Provider documents, media and controlled uploads
Ingress/CDN	TLS termination, routing and edge protection
External services	Maps, payment, OTP/SMS, push/email

4. Access & Operational Safety

- Use MFA for privileged accounts.
- Do not share production credentials.
- Use named accounts for human operators.
- Use break-glass access only for genuine emergencies and record the reason.
- Never paste secrets, tokens, OTPs or payment credentials into tickets/chats.
- Use read-only access for investigation whenever possible.
- Record significant operational actions in the incident/change record.

4.1 Operational Roles

Role	Primary responsibility
Incident Commander	Coordinates incident, priorities and decisions
Technical Lead	Leads diagnosis and mitigation
DevOps	Infrastructure, deployment and recovery actions
Backend/Frontend	Application diagnosis and fixes
QA	Validation and regression/smoke verification
Security	Security incident assessment and containment
Support/Ops	Customer/provider communication and case handling

5. Standard Start-of-Day Health Check

Perform or automate the following checks before high-volume operating periods:

- Application availability and HTTP error rate are normal.
- Database connectivity, capacity, locks and connection pool are healthy.
- Redis latency, memory and errors are normal.
- Worker queues are processing without abnormal backlog.
- WebSocket connections and event delivery are healthy.
- Payment and notification integrations show no unresolved failures.
- Storage capacity and upload/download error rates are normal.
- No unresolved critical alerts or active incidents exist.

Suggested operational dashboard:

Availability | 5xx | p95/p99 latency | DB connections | DB CPU/storage | Redis memory/latency | queue depth | worker failures | WebSocket connections | booking success | provider acceptance | payment failures

6. Standard Deployment Procedure

6.1 Before Deployment

- Confirm release version/commit and change scope.
- Confirm CI, security and QA gates are passing.
- Review database migration risk.
- Verify backup health for high-risk releases.
- Confirm rollback artifact is available.

- Check for active incidents or major traffic events.
- Communicate deployment window when customer impact is possible.

6.2 Deploy

BUILD/SELECT RELEASE → DEPLOY STAGING → SMOKE TEST → APPROVE → DEPLOY PRODUCTION → HEALTH CHECK → SYNTHETIC TEST → OBSERVE

6.3 After Deployment

- Verify health/readiness endpoints.
- Check application errors and latency.
- Verify booking creation and provider assignment.
- Verify realtime status/location events.
- Verify notifications where relevant.
- Check worker queue processing.
- Check payment status/reconciliation if the release touches billing.
- Keep elevated monitoring for the agreed observation window.

7. Rollback Procedure

Use rollback when the release causes material availability, integrity, security or performance degradation.

7.1 Procedure

1. Declare/record deployment issue → 2. Stop further rollout → 3. Identify last known-good version → 4. Roll back application → 5. Verify health → 6. Validate critical journeys → 7. Monitor → 8. Record root cause

Database warning: Do not automatically reverse migrations. If the database has already changed, prefer a compatible forward fix unless a tested rollback is explicitly safe.

8. Incident Severity

Severity	Definition	Examples
SEV-1	Critical customer/business impact or safety/integrity risk	Platform unavailable; widespread booking failure; payment corruption; major security event
SEV-2	Major degradation with significant operational impact	High error rate; matching unavailable in a region; realtime tracking broadly broken
SEV-3	Limited impact or workaround exists	Specific feature degraded; provider notification delays
SEV-4	Minor/non-urgent issue	Cosmetic issue; low-risk operational defect

9. Incident Response Procedure

9.1 Detect & Acknowledge

- Confirm alert or report is real.
- Record start time and affected component.
- Assign incident severity.
- Open an incident record and assign Incident Commander for SEV-1/SEV-2.

9.2 Triage

- Determine blast radius: customers, providers, regions, bookings and payments.

- Identify recent deployments or infrastructure changes.
- Check application logs using correlation/request IDs.
- Check database, Redis, worker, WebSocket and external dependency health.
- Determine whether the issue is availability, correctness, performance or security.

9.3 Mitigate

- Prefer reversible mitigations.
- Disable affected feature using a feature flag if available.
- Reduce load or stop non-critical background work if it threatens core operations.
- Rollback a faulty release when appropriate.
- Fail safely rather than creating inconsistent booking/payment state.

9.4 Recover & Verify

- Confirm errors and latency return toward baseline.
- Validate customer and provider critical journeys.
- Check queues for stuck/duplicate jobs.
- Check booking state consistency.
- Check payment/reconciliation state if financial flows were involved.
- Keep monitoring until stability is established.

10. Application Health Troubleshooting

Symptom	First checks	Likely areas
5xx spike	Recent release, API logs, dependency health	API, DB, Redis, external services
High latency	p95/p99, DB queries, CPU, pool usage	DB, API, external APIs
No provider matching	Queue, provider availability, geospatial queries	Matching, Redis, PostGIS
Booking stuck	State history, worker events, locks	Booking state machine, worker, realtime
Notifications delayed	Queue depth, provider status, integration errors	Worker, SMS/push/email
Tracking not updating	WebSocket connections, location writes	Realtime, Redis, mobile/network
Uploads failing	Storage status, signed URL, size/type limits	Object storage/API

11. Database Operations

11.1 Safe Investigation

- Prefer read-only queries.
- Check connection utilization and long-running transactions.
- Check locks before restarting or terminating sessions.
- Identify whether a problem is query-level, connection-level or infrastructure-level.
- Avoid ad-hoc mass updates during incidents.

11.2 Database Incident Checklist

- Database reachable?
- Storage capacity sufficient?
- Connection pool saturated?
- CPU/memory/IO abnormal?
- Long-running transactions?
- Lock contention?
- Recent migration?
- Replication/backup status healthy where applicable?

12. Redis Operations

- Check memory usage, eviction behavior, latency and connection errors.
- Identify queue backlog separately from cache pressure.
- Do not delete Redis data blindly during an incident.
- If Redis is lost, application must recover critical state from PostgreSQL and rebuild ephemeral state where designed.
- After recovery, verify queues, locks and realtime coordination.

13. Worker & Queue Operations

Condition	Action
Queue growing	Check worker health, dependency failures and throughput
Repeated job failures	Inspect retry/error reason; stop poison-message loop if required
Duplicate jobs	Check idempotency keys and worker concurrency
Stuck job	Inspect lease/lock/timeout behavior
Dead-letter items	Review individually before replay
Notification backlog	Prioritize transactional notifications and fix integration bottleneck

All business-critical jobs should be idempotent where duplicate execution is possible.

14. Booking State Recovery

RoadResQ booking state should be treated as a controlled state machine. Investigate the status history and authoritative database records before changing a state.

Typical lifecycle: REQUESTED → MATCHING → ACCEPTED → EN_ROUTE → ARRIVED → INSPECTION → ESTIMATE_PENDING → APPROVAL_PENDING → IN_PROGRESS → COMPLETED → PAYMENT_PENDING/PAID → REVIEWED

- Never skip directly to PAID without a valid financial record.
- Never mark a provider ARRIVED without the expected arrival verification mechanism where required.
- Use audited admin recovery actions for exceptional state corrections.
- Record why a state was corrected and who performed the action.

15. Realtime / Tracking Operations

- Authenticate WebSocket connections.
- Verify the user/provider is authorized to receive the booking's events.
- Check connection counts and event delivery errors.

- On reconnect, clients should request/reconcile authoritative booking state.
- Location updates should be rate-limited and stored according to approved retention rules.
- If realtime is degraded, booking APIs remain the source of truth and the UI should show a degraded/offline state.

16. Payment Operations

Payments require correctness over speed. Treat the payment gateway and RoadResQ records as separate systems that must be reconciled.

- Never trust client-reported payment success.
- Validate gateway webhooks securely.
- Use idempotency keys for payment creation/retry flows.
- Record gateway transaction/reference IDs.
- Reconcile pending/failed/unknown transactions.
- Do not manually mark a transaction paid without evidence.
- For discrepancies, create a support/finance case and use controlled correction workflows.

16.1 Payment Incident

PAYMENT ISSUE → IDENTIFY TRANSACTION → CHECK ROADRESQ RECORD → CHECK GATEWAY STATUS → VERIFY WEBHOOK/REFERENCE → RECONCILE → UPDATE VIA CONTROLLED FLOW

17. Provider Verification Operations

- Review identity/business/service documents according to approved verification policy.
- Do not expose provider documents to unauthorized staff.
- Use states PENDING → VERIFIED → ACTIVE or REJECTED/SUSPENDED as applicable.
- Record reviewer and decision reason.
- Suspend providers when safety, fraud or policy concerns require immediate containment.
- Provider status changes should be auditable.

18. Customer/Provider Support Operations

Issue	Operational response
Provider no-show	Check booking status, provider actions, timestamps and contact attempts; re-dispatch if policy allows
Customer cancellation	Verify state and applicable cancellation policy before financial adjustment
Wrong estimate	Review inspection/estimate history and approval evidence
Disputed completion	Check job card, status history, timestamps, photos and communication records
Payment dispute	Check invoice, payment and gateway reconciliation records
Fraud/safety report	Preserve evidence, restrict access, escalate to security/operations

19. Security Incident Runbook

Examples: compromised account, leaked secret, unauthorized provider access, suspicious payment activity, malicious upload, abnormal API traffic or data exposure.

Immediate response:

- Contain the affected account/service/key.
- Rotate compromised credentials where required.
- Block malicious traffic or access.
- Preserve logs and relevant evidence.
- Assess affected data and users.
- Notify appropriate internal owners and follow applicable legal/privacy requirements.
- Do not destroy evidence during cleanup.

If a production secret is suspected to be exposed, treat it as compromised until rotation and verification are complete.

20. Backup & Restore Operations

20.1 Backup Verification

- Confirm scheduled backups completed successfully.
- Check retention and storage health.
- Verify backup encryption/access controls.
- Monitor failed backup jobs.

20.2 Restore Drill

SELECT RECOVERY POINT → RESTORE ISOLATED ENVIRONMENT → VALIDATE SCHEMA/DATA → TEST APPLICATION → RECORD RTO/RPO → DOCUMENT GAPS

Restore testing should be performed periodically rather than assuming backups are usable.

21. Disaster Recovery

Scenario	Priority action
Application outage	Stabilize/rollback/redeploy healthy artifact
Database outage	Follow managed DB failover/recovery procedure
Redis outage	Restore ephemeral services and rebuild safe state
Storage outage	Follow storage provider recovery/failover process
Region/cloud outage	Execute approved DR architecture and traffic recovery plan
Data corruption	Stop destructive processes, preserve evidence, restore/reconcile from known-good point

RPO and RTO targets must be set by the business and validated through drills.

22. Scheduled Maintenance

- Dependency/security updates.
- Database maintenance and index review.
- Log/temporary data cleanup.
- Backup restore drills.
- Certificate/domain checks.
- Cloud resource and cost review.
- Third-party API quota review.

- Access review and credential rotation.

Use a maintenance/change record for production-impacting work.

23. Monitoring & Alert Thresholds

Exact thresholds should be calibrated from real production baselines. Initial alerts should cover:

- Availability below agreed SLO.
- 5xx/error rate above baseline.
- p95/p99 latency above service target.
- Database CPU/storage/connections approaching safe limits.
- Redis memory/latency/eviction anomalies.
- Queue backlog or retry rate increasing.
- WebSocket connection/event failure spikes.
- Payment webhook/reconciliation failures.
- Backup failures.
- Certificate or domain expiry risk.

24. Operational SLOs

SLO category	Target definition
Availability	Agreed percentage for critical API/customer journeys
Latency	Agreed p95/p99 targets for key APIs
Booking reliability	High percentage of valid requests successfully matched/completed
Realtime	Agreed event delivery/update latency for active jobs
Payment reliability	Correctness/reconciliation over raw response speed
Recovery	RTO/RPO targets defined for critical data/services

Final numeric targets should be approved after baseline performance testing and business risk review.

25. Feature Flags & Safe Degradation

- Use feature flags for risky or staged features where possible.
- Keep a safe fallback path for maps, notifications and realtime where feasible.
- Disable non-critical features before compromising core booking operations.
- Record who changed production flags and why.
- Remove obsolete flags after rollout.

26. Change Management

Change type	Control
Routine	Standard reviewed deployment
High-risk	Explicit approval + backup/rollback validation
Emergency	Immediate mitigation allowed; retrospective review required

Change type	Control
Infrastructure	IaC/review preferred; emergency changes reconciled into code
Database	Migration review + staging validation

27. Incident Communication

- Use one authoritative incident record/channel.
- Communicate impact, current status, mitigation and next update—not speculation.
- Customer/provider-facing communication should be accurate and privacy-safe.
- Avoid exposing internal security details or sensitive information.
- After resolution, publish an appropriate internal summary and action items.

28. Post-Incident Review

Section	Required content
Summary	What happened and duration
Impact	Customers/providers/bookings/payments affected
Timeline	Detection, decisions, mitigation, recovery
Root cause	Technical/process causes
Contributing factors	Monitoring, deployment, dependency or human factors
What worked	Successful controls
What failed	Gaps
Actions	Owner + priority + due date

29. Daily / Weekly / Monthly Operations

Cadence	Checks
Daily	Health dashboards, critical alerts, backups, queues, payment/reconciliation exceptions
Weekly	Dependency/security review, error trends, capacity, unresolved incidents, failed jobs
Monthly	Access review, backup restore/DR drill where scheduled, cost review, SLO review, certificate/domain review
Quarterly	Full disaster recovery exercise, threat/security review, runbook validation, architecture/capacity review

30. Emergency Quick Reference

When production is broken:

1. Protect users and stop unsafe actions. 2. Check whether a recent release caused the issue. 3. Check API/DB/Redis/worker/realtime health. 4. Determine blast radius. 5. Mitigate with the safest reversible action. 6. Roll back when justified. 7. Verify booking/payment integrity. 8. Communicate status. 9. Monitor recovery. 10. Record and review the incident.

31. Production Operational Checklist

Area	Ready?
Application health/readiness checks	■
Monitoring and alerting	■
CI/CD and rollback	■
Database backups + restore drill	■
Redis/queue monitoring	■
WebSocket/realtime monitoring	■
Payment reconciliation	■
Provider verification controls	■
Security incident process	■
Support/dispute process	■
Access/MFA/audit logging	■
Incident communication process	■
Runbook reviewed against live infrastructure	■

32. Definition of Done — Production Operations

- Operators can identify the health of every critical production component.
- Standard deployment and rollback procedures are documented and tested.
- Critical incidents have severity, ownership and escalation paths.
- Database, Redis, workers and realtime have troubleshooting procedures.
- Payment reconciliation and booking-state recovery are controlled.
- Backups are monitored and restore drills are performed.
- Security incidents have containment and evidence-preservation procedures.
- Post-incident reviews create owned follow-up actions.
- Runbook procedures match the actual deployed architecture.

33. Final Operational Standard

RoadResQ production operations should be predictable, auditable and recovery-oriented. The goal is not merely to keep servers running; it is to preserve safe customer assistance, reliable provider dispatch, correct booking state, trustworthy payments and controlled handling of sensitive data even when individual components fail.